

Cahier des charges – Plateforme CloudDelivery

1. Informations générales

- **Nom du projet** : CloudDelivery
- **Type de projet** : Application SaaS distribuée (microservices) + plateforme DevOps complète
- **Domaine** : Gestion de livraisons pour e-commerce / restauration
- **Objectif principal** : Concevoir et déployer une plateforme de gestion des commandes et livraisons, entièrement automatisée via Infrastructure as Code (Terraform + Ansible) et orchestrée sur Kubernetes.

1.1. Parties prenantes

- **Client / Commanditaire** : Organisation fictive « CloudDelivery »
 - **Équipe de développement** :
 - Développeurs Backend (Spring Boot, Node.js)
 - Développeurs Frontend (Angular)
 - DevOps / Infra (Docker, K8s, Terraform, Ansible, Jenkins)
 - **Utilisateurs finaux** :
 - Clients finaux (e-commerçants et leurs clients)
 - Administrateurs / opérateurs logistiques
 - Livreurs
-

2. Objectifs du projet

2.1. Objectifs fonctionnels

- Permettre la **création, gestion et suivi** de commandes.
- Gérer les **livraisons** (affectation livreurs, suivi temps réel, statuts).
- Offrir un **portail client** pour suivre les colis.
- Offrir un **portail admin** pour la gestion opérationnelle (livreurs, zones, KPIs).
- Proposer un **système de notifications** (temps réel via WebSockets).

2.2. Objectifs techniques

- Architecture **microservices** :
 - Backend Spring Boot (Auth, Orders, Deliveries)
 - Gateway temps réel Node.js
 - Frontend Angular (portail client + admin)
 - Déploiement **conteneurisé** (Docker).
 - Orchestration via **Kubernetes** (AKS/EKS/GKE ou équivalent).
 - Exposition via **Nginx** (Ingress Controller).
 - CI/CD complet avec **Jenkins**.
 - **Infrastructure as Code** :
 - Provisionnement Cloud via **Terraform**.
 - Configuration serveurs et services via **Ansible**.
-

3. Périmètre fonctionnel

3.1. Acteurs

- **Client** : passe commande, consulte l'historique et suit la livraison.
- **Admin / Opérateur** : gère les livreurs, zones, commandes et livraisons.
- **Livreur** : consulte ses tournées et met à jour le statut / position.
- **Système externe (API)** : optionnel, pour intégration e-commerce.

3.2. Fonctionnalités – Portail Client

- Création de compte / authentification.
- Consultation de l'historique des commandes.
- Suivi temps réel d'une commande :
 - Visualisation sur carte (position livreur).
 - Timeline statuts (PREPARING, ON_ROAD, DELIVERED...).
- Téléchargement de facture (optionnel).

3.3. Fonctionnalités – Portail Admin

- Dashboard :
 - Nombre de commandes / livraisons (par jour/semaine/mois).
 - Temps moyen de livraison.
 - Taux de succès / échecs.

- Gestion des livreurs :
 - CRUD livreurs (identité, zone, statut).
- Gestion des commandes :
 - Consultation des commandes / filtres.
- Gestion des livraisons :
 - Consultation globale.
 - Affectation d'un livreur à une commande.

3.4. Fonctionnalités – Livreurs

- Authentification.
- Liste des livraisons du jour.
- Mise à jour des statuts :
 - PICKED_UP, ON_ROAD, DELIVERED, FAILED.
- Envoi régulier de la position GPS.

3.5. Fonctionnalités transverses

- **Authentification & Autorisation** :
 - JWT (roles : ADMIN, CLIENT, DRIVER).
- **Temps réel** :
 - WebSockets / Socket.io pour pousser les mises à jour de position et de statut.
- **Logging et traçabilité** :
 - Logging centralisé des requêtes et événements.

4. Périmètre technique

4.1. Technologies principales

- **Frontend** : Angular (TypeScript)
- **Backend** :
 - Spring Boot (Java) – microservices : `auth-service`, `order-service`, `delivery-service`
 - Node.js (NestJS ou Express) – `realtime-gateway`
- **Base de données** : PostgreSQL
- **Reverse proxy / Ingress** : Nginx
- **Conteneurs** : Docker
- **Orchestration** : Kubernetes (AKS/EKS/GKE ou K3d/kind en dev)
- **CI/CD** : Jenkins
- **Infra as Code** :
 - Terraform (provisionnement Cloud)
 - Ansible (configuration serveurs / Jenkins / Nginx)

5. Architecture applicative

5.1. Microservices

- **Auth Service** (Spring Boot)
 - Gestion utilisateurs, rôles, JWT, refresh tokens.
- **Order Service** (Spring Boot)
 - Gestion commandes, clients, articles.
- **Delivery Service** (Spring Boot)
 - Gestion livraisons, livreurs, événements de livraison.
- **Realtime Gateway** (Node.js)
 - Gestion des WebSockets / Socket.io.
 - Réception d'événements depuis Delivery Service.
 - Diffusion aux fronts Angular.

5.2. Frontend

- **Customer Portal** (Angular)
 - Pour clients finaux.
- **Admin Portal** (Angular)
 - Pour administrateurs / opérateurs.

6. Architecture système distribuée (vue logique)

- Les frontends Angular consomment :
 - API REST des microservices Spring Boot (via Nginx/Ingress).
 - WebSocket vers le Realtime Gateway.
- Les microservices communiquent entre eux via HTTP/REST interne.
- Les services applicatifs sont entièrement stateless ; l'état persistant est dans PostgreSQL.

- Les services sont packagés en conteneurs Docker et déployés sur Kubernetes, chaque service ayant :
 - Son Deployment K8s.
 - Son Service K8s (ClusterIP).
- Accès entrants via Nginx Ingress Controller, exposé par un Load Balancer.

7. Infrastructure as Code – Spécifications détaillées

Partie prioritaire – La plus importante du cahier des charges.

7.1. Principes généraux IaC

- **Reproductibilité** : Toute l'infrastructure doit être recréable à partir du code (Terraform + Ansible).
- **Idempotence** :
 - Terraform : appliquer plusieurs fois ne doit pas changer l'état.
 - Ansible : les playbooks doivent être idempotents.
- **Séparation des responsabilités** :
 - Terraform : provisionnement des ressources Cloud.
 - Ansible : configuration logicielle sur les ressources (VM Jenkins, Nginx, etc.).
- **Séparation des environnements** :
 - Environnements minimum : dev , prod .
 - Éventuellement stage selon besoin.

7.2. Terraform – Spécifications

7.2.1. Structure du code Terraform

- Organisation recommandée :

```
infra/  
  terraform/  
    envs/  
      dev/  
        main.tf  
        variables.tf  
        outputs.tf  
        backend.tf  
      prod/  
        main.tf  
        variables.tf  
        outputs.tf  
        backend.tf  
    modules/  
      network/  
        main.tf  
        variables.tf  
        outputs.tf  
      k8s_cluster/  
        main.tf  
        variables.tf  
        outputs.tf  
      postgres/  
        main.tf  
        variables.tf  
        outputs.tf  
      registry/  
        main.tf  
        variables.tf  
        outputs.tf  
      jenkins_vm/ (optionnel)  
        main.tf  
        variables.tf  
        outputs.tf
```

